

指向 `s.nameValue` 所指的那个 `string` 吗? 也就是说 `reference` 自身可被改动吗? 如果是, 那可就开辟了新天地, 因为 C++ 并不允许“让 `reference` 改指向不同对象”。换一个想法, `p.nameValue` 所指的那个 `string` 对象该被修改, 进而影响“持有 `pointers` 或 `references` 而且指向该 `string`”的其他对象吗? 也就是对象不被直接牵扯到赋值操作内? 编译器生成的 `copy assignment` 操作符究竟该怎么做呢?

面对这个难题, C++ 的响应是拒绝编译那一行赋值动作。如果你打算在一个“内含 `reference` 成员”的 `class` 内支持赋值操作 (`assignment`), 你必须自己定义 `copy assignment` 操作符。面对“内含 `const` 成员” (如本例之 `objectValue`) 的 `classes`, 编译器的反应也一样。更改 `const` 成员是不合法的, 所以编译器不知道如何在它自己生成的赋值函数内面对它们。最后还有一种情况: 如果某个 `base classes` 将 `copy assignment` 操作符声明为 `private`, 编译器将拒绝为其 `derived classes` 生成一个 `copy assignment` 操作符。毕竟编译器为 `derived classes` 所生的 `copy assignment` 操作符想象中可以处理 `base class` 成分 (见条款 12), 但它们当然无法调用 `derived class` 无权调用的成员函数。编译器两手一摊, 无能为力。

请记住

- 编译器可以暗自为 `class` 创建 `default` 构造函数、`copy` 构造函数、`copy assignment` 操作符, 以及析构函数。

条款 06: 若不想使用编译器自动生成的函数, 就该明确拒绝

Explicitly disallow the use of compiler-generated functions you do not want.

地产中介商卖的是房子, 一个中介软件系统自然而然想必有个 `class` 用来描述待售房屋:

```
class HomeForSale { ... };
```

每一位真正的地产中介商都会说, 任何一笔资产都是天上地下独一无二, 没有两笔完全相像。因此我们也认为, 为 `HomeForSale` 对象做一份副本有点没道理。你怎么可以复制某些先天独一无二的东西呢? 因此, 你应该乐意看到 `HomeForSale` 的对象拷贝动作以失败收场:

```
HomeForSale h1;
HomeForSale h2;
HomeForSale h3(h1);           //企图拷贝 h1 — 不该通过编译
h1 = h2;                       //企图拷贝 h2 — 也不该通过编译
```

啊呀，阻止这一类代码的编译并不是很直观。通常如果你不希望 `class` 支持某一特定机能，只要不声明对应函数就是了。但这个策略对 `copy` 构造函数和 `copy assignment` 操作符却不起作用，因为条款 5 已经指出，如果你不声明它们，而某些人尝试调用它们，编译器会为你声明它们。

这把你逼到了一个困境。如果你不声明 `copy` 构造函数或 `copy assignment` 操作符，编译器可能为你产出一份，于是你的 `class` 支持 `copying`。如果你声明它们，你的 `class` 还是支持 `copying`。但这里的目标却是要阻止 `copying`！

答案的关键是，所有编译器产出的函数都是 `public`。为阻止这些函数被创建出来，你得自行声明它们，但这里并没有什么需求使你必须将它们声明为 `public`。因此你可以将 `copy` 构造函数或 `copy assignment` 操作符声明为 `private`。藉由明确声明一个成员函数，你阻止了编译器暗自创建其专属版本；而令这些函数为 `private`，使你得以成功阻止人们调用它。

一般而言这个做法并不绝对安全，因为 `member` 函数和 `friend` 函数还是可以调用你的 `private` 函数。除非你够聪明，不去定义它们，那么如果某些人不慎调用任何一个，会获得一个连接错误（`linkage error`）。“将成员函数声明为 `private` 而且故意不实现它们”这一伎俩是如此为大家接受，因而被用在 C++ `iostream` 程序库中阻止 `copying` 行为。是的，看看你手上的标准程序库实现码中的 `ios_base`, `basic_ios` 和 `sentry`。你会发现无论哪一个，其 `copy` 构造函数和 `copy assignment` 操作符都被声明为 `private` 而且没有定义。

将这个伎俩施行于 `HomeForSale` 也很简单：

```
class HomeForSale {
public:
    ...
private:
    ...
    HomeForSale(const HomeForSale&);           //只有声明
    HomeForSale& operator=(const HomeForSale&);
};
```

或许你注意到了，我没写函数参数的名称。唔，参数名称并非必要，只不过大家总是习惯写出来。这个函数毕竟不会被实现出来，也很少被使用，指定参数名称又有何用？

有了上述 `class` 定义，当客户企图拷贝 `HomeForSale` 对象，编译器会阻挠他。如果你不慎在 `member` 函数或 `friend` 函数之内那么做，轮到连接器发出抱怨。

将连接期错误移至编译期是可能的(而且那是好事, 毕竟愈早侦测出错误愈好), 只要将 *copy* 构造函数和 *copy assignment* 操作符声明为 `private` 就可以办到, 但不是 `HomeForSale` 自身, 而是在一个专门为了阻止 *copying* 动作而设计的 `base class` 内。这个 `base class` 非常简单:

```
class Uncopyable {
protected:                                //允许 derived 对象构造和析构
    Uncopyable() {}
    ~Uncopyable() {}
private:
    Uncopyable(const Uncopyable&);          //但阻止 copying
    Uncopyable& operator=(const Uncopyable&);
};
```

为求阻止 `HomeForSale` 对象被拷贝, 我们唯一需要做的就是继承 `Uncopyable`:

```
class HomeForSale: private Uncopyable {    //class 不再声明
    ...                                    //copy 构造函数或
};                                          //copy assign. 操作符
```

这行得通, 因为只要任何人——甚至是 `member` 函数或 `friend` 函数——尝试拷贝 `HomeForSale` 对象, 编译器便试着生成一个 *copy* 构造函数和一个 *copy assignment* 操作符, 而正如条款 12 所说, 这些函数的“编译器生成版”会尝试调用其 `base class` 的对应兄弟, 那些调用会被编译器拒绝, 因为其 `base class` 的拷贝函数是 `private`。

`Uncopyable class` 的实现和运用颇为微妙, 包括不一定得以 `public` 继承它(见条款 32 和 39), 以及 `Uncopyable` 的析构函数不一定得是 `virtual`(见条款 7)等等。`Uncopyable` 不含数据, 因此符合条款 39 所描述的 `empty base class optimization` 资格。但由于它总是扮演 `base class`, 因此使用这项技术可能导致多重继承(译注: 因为你往往还可能需要继承其他 `class`)(多重继承见条款 40), 而多重继承有时会阻止 `empty base class optimization`(再次见条款 39)。通常你可以忽略这些微妙点, 只像上面那样使用 `Uncopyable`, 因为它完全像“广告”所说的能够正确运作。也可以使用 `Boost`(见条款 55)提供的版本, 那个 `class` 名为 `noncopyable`, 是个还不错的家伙, 我只是认为其名称有点……呃……不太自然。

请记住

- 为驳回编译器自动(暗自)提供的机能, 可将相应的成员函数声明为 `private` 并且不予实现。使用像 `Uncopyable` 这样的 `base class` 也是一种做法。

条款 07: 为多态基类声明 virtual 析构函数

Declare destructors virtual in polymorphic base classes.

有许多种做法可以记录时间, 因此, 设计一个 `TimeKeeper` base class 和一些 `derived classes` 作为不同的计时方法, 相当合情合理:

```
class TimeKeeper {
public:
    TimeKeeper();
    ~TimeKeeper();
    ...
};
class AtomicClock: public TimeKeeper { ... }; //原子钟
class WaterClock: public TimeKeeper { ... }; //水钟
class WristWatch: public TimeKeeper { ... }; //腕表
```

许多客户只想在程序中使用时间, 不想操心时间如何计算等细节, 这时候我们可以设计 **factory** (工厂) 函数, 返回指针指向一个计时对象。Factory 函数会“返回一个 base class 指针, 指向新生成之 derived class 对象”:

```
TimeKeeper* getTimeKeeper();           //返回一个指针, 指向一个
                                       //TimeKeeper 派生类的动态分配对象
```

为遵守 **factory** 函数的规矩, 被 `getTimeKeeper()` 返回的对象必须位于 **heap**。因此为了避免泄漏内存和其他资源, 将 **factory** 函数返回的每一个对象适当地 `delete` 掉很重要:

```
TimeKeeper* ptk = getTimeKeeper();     //从 TimeKeeper 继承体系
                                       //获得一个动态分配对象。
...                                     //运用它...
delete ptk;                             //释放它, 避免资源泄漏。
```

条款 13 说“倚赖客户执行 `delete` 动作, 基本上便带有某种错误倾向”, 条款 18 则谈到 **factory** 函数接口该如何修改以便预防常见之客户错误, 但这些在此都是次要的, 因为此条款内我们要对付的是上述代码的一个更根本弱点: 纵使客户把每一件事都做对了, 仍然没办法知道程序如何行动。

问题出在 `getTimeKeeper` 返回的指针指向一个 `derived class` 对象 (例如 `AtomicClock`), 而那个对象却经由一个 `base class` 指针 (例如一个 `TimeKeeper*` 指针) 被删除, 而目前的 `base class` (`TimeKeeper`) 有个 **non-virtual** 析构函数。